

SC102 L06：程序化物理：碰撞盒生成——详细学习笔记

目录

1	课程导览与物理单元概述	3
2	碰撞检测算法基础	3
2.1	碰撞检测的任务与意义	3
2.2	基本几何体 (Primitives)	4
3	矩形与矩形的碰撞检测	4
3.1	算法原理	4
3.2	代码实现	5
3.3	算法复杂度分析	5
3.4	扩展：重叠深度计算	6
4	线段与三角形的相交检测	6
4.1	问题描述与应用场景	6
4.2	求线段与平面的交点	6
4.3	2D 中的点是否在三角形内：面积法	7
4.4	3D 中的点是否在三角形内：重心坐标法	8
4.5	完整流程：线段与三角形的 3D 相交检测	10
5	线段与模型的相交检测	11
5.1	模型即三角形的集合	11
5.2	性能问题	11
6	三角形与三角形的碰撞检测	11
6.1	问题描述	11
7	包围体积层级 (BVH)	12
7.1	为什么需要 BVH	12

7.2	BVH 的工作原理	12
7.3	射线与 BVH 的相交检测：伪代码	13
7.4	Octree 八叉树	14
7.5	K-D Tree	14
8	Unity LineRenderer 组件	15
8.1	组件概述	15
8.2	核心属性：Positions	15
8.3	LineRenderer 的实用属性	16
9	Unity PolygonCollider2D 组件	17
9.1	组件概述	17
9.2	核心属性：Points	17
9.3	LineRenderer 与 PolygonCollider2D 的结合	17
10	从线段生成贴合碰撞器	17
10.1	核心问题	17
10.2	算法：通过线段扩展生成多边形	18
10.3	完整代码实现	18
10.4	拐角处理详解	20
10.5	运行时实时更新	20
11	课后习题详解	21
11.1	习题 1：使用叉乘判断点是否在三角形内	21
11.2	习题 2：圆与矩形的碰撞检测	23
11.3	习题 3：线段点集的性能优化	25
12	总结	27

1 课程导览与物理单元概述

本课是 SC102 Unity 游戏开发课程第二单元（物理与动画）的第二讲，主题为「程序化物理：碰撞盒生成」。该单元涵盖以下主题 [p.2]：

- **L05:** Unity 物理系统详解——连接关节 (Joints)、2D 作用器 (Effectors)、各类射线检测 (Box Cast、Sphere Cast)、连续碰撞检测 (CCD)
- **L06:** 程序化物理：碰撞盒生成——碰撞检测算法、优化结构 (Octree、K-D Tree)、LineRenderer、PolygonCollider2D、根据线段自动生成碰撞形状
- **L07:** 程序化生成——程序化网格 (Procedural Mesh)、草的生成
- **L08:** 软体模拟——质点弹簧系统 (Mass-Spring System)、布料 (Cloth) 等软体物理
- **L09:** 动画曲线与 Tweening ——常见缓动曲线、曲线采样器、动画过渡工具类
- **L10:** 程序化动画：反向运动学 (Inverse Kinematics, IK) 及其数学基础

本课聚焦于「程序化物理」的核心内容 [p.3]：

1. 基本几何体的碰撞检测算法 (Primitive Collision Detection)
2. 优化结构：包围体积层级 BVH (Octree、K-D Tree、BSP)
3. Unity LineRenderer 组件
4. Unity PolygonCollider2D 组件
5. 实现从线段自动生成贴合物理形状

2 碰撞检测算法基础

2.1 碰撞检测的任务与意义

碰撞检测 (Collision Detection) 是物理引擎 (Physics Engine) 的核心任务之一。它的目标是判断两个物体在空间中是否发生重叠 (Overlap)，如果发生重叠，则进一步计算碰撞点 (Contact Point) 和碰撞法线 (Contact Normal) [p.4]。

虽然在课程中我们并不会真正实现一个功能完备的物理引擎，但掌握基本几何体之间的碰撞检测算法仍然具有重要的实用价值：

1. **实际应用场景:** 在游戏开发中，我们经常遇到需要判断空间交叠的问题。例如：判断一个 UI 元素是否被另一个覆盖、判断角色是否进入了某个区域、判断子弹是否命中了目标。这些场景本质上都是碰撞检测问题。
2. **数学思想训练:** 碰撞检测算法涉及大量的几何计算 (向量运算、叉乘、点乘、重心坐标、平面方程等)，掌握这些算法有助于加深对计算机图形学和计算几何的理解，培养在多维空间中进行数学推理的能力。

2.2 基本几何体 (Primitives)

基本的碰撞检测算法专门处理**基本几何体 (Primitives)**之间的重叠判断。常见的几何体包括 [p.4]:

- **三角形 (Triangle)**: 最基本的渲染图元, 所有 3D 模型的表面都由三角形构成。三角形碰撞检测是网格碰撞检测的基础。
- **球体 (Sphere)**: 最简单、最高效的包围体。球-球碰撞检测只需要比较球心距离与半径之和。广泛用于粗略的碰撞剔除 (Broad Phase)。
- **轴对齐包围盒 AABB (Axis-Aligned Bounding Box)**: 各边与坐标轴平行的长方体。AABB 的碰撞检测效率极高, 因为只需要进行区间重叠判断。Unity 中的 BoxCollider 本质上就是 AABB (在 3D 中) 或其 2D 对应物。
- **圆柱/圆锥 (Cylinder/Cone)**: 用于近似人形角色或特定形状的物体。Unity 中的 CapsuleCollider 可以视为圆柱两端加半球的组合。
- **胶囊体 (Capsule)**: 由圆柱体和两个半球组成, 是 Unity CharacterController 的默认碰撞体。
- **定向包围盒 OBB (Oriented Bounding Box)**: 可以旋转的长方体, 比 AABB 更紧密地包围物体, 但检测更复杂。

理解这些基本几何体之间的碰撞检测是深入掌握物理引擎工作原理的基石。以下各节将逐一展开最常见和最重要的检测算法。

3 矩形与矩形的碰撞检测

3.1 算法原理

矩形之间的碰撞检测是最简单、最直观的碰撞检测算法之一。这也是为什么在实际游戏开发中强烈推荐使用 BoxCollider (本质上是矩形/长方体碰撞器) 的原因 [p.5]。

在 2D 空间中, 两个轴对齐矩形 (Axis-Aligned Rectangle) 不重叠的充要条件是: 其中一个矩形完全位于另一个矩形的左侧、右侧、上方或下方。换句话说, 如果两个矩形在 X 轴上的投影区间有重叠, 且同时在 Y 轴上的投影区间也有重叠, 则它们发生了碰撞。

形式化地, 设有两个矩形 R_1 和 R_2 , 其中 R_1 的左边界为 X_1 、右边界为 $X_1 + W_1$, 下边界为 Y_1 、上边界为 $Y_1 + H_1$ (R_2 同理)。则:

- 不相交 (Separated) 的条件:

$$R_1.Right < R_2.X \quad \text{或} \quad R_2.Right < R_1.X \quad \text{或} \quad R_1.Bottom < R_2.Y \quad \text{或} \quad R_2.Bottom < R_1.Y$$

- 相交 (Intersecting) 的条件: 以上四个条件均不成立。

这种判断方法利用了**分离轴定理 (Separating Axis Theorem, SAT)**的最简单形式。SAT 指出: 如果存在一条轴, 使得两个凸形在该轴上的投影不重叠, 则这两个凸形不相交。对于轴对齐矩形, X 轴和 Y 轴就是两条足够用的分离轴。

3.2 代码实现

首先定义一个简单的矩形数据结构，包含位置、宽高以及便捷的边界属性：

Listing 1: 矩形数据结构 (p.5)

```
struct Rectangle
{
    public float X { get; set; }
    public float Y { get; set; }
    public float Width { get; set; }
    public float Height { get; set; }
    // 便捷属性：右边界
    public float Right { get { return X + Width; } }
    // 便捷属性：上边界（注意：Unity 中使用 Bottom 表示底边）
    public float Bottom { get { return Y + Height; } }

    public Rectangle(float x, float y, float width, float height)
    {
        X = x; Y = y; Width = width; Height = height;
    }
}
```

核心碰撞检测函数：

Listing 2: 矩形碰撞检测函数 (p.5)

```
static bool DoRectanglesIntersect(Rectangle r1, Rectangle r2)
{
    // 检查 r1 是否完全在 r2 的左侧
    if (r1.Right < r2.X) return false;
    // 检查 r2 是否完全在 r1 的左侧
    if (r2.Right < r1.X) return false;
    // 检查 r1 是否完全在 r2 的下方
    if (r1.Bottom < r2.Y) return false;
    // 检查 r2 是否完全在 r1 的下方
    if (r2.Bottom < r1.Y) return false;
    // 以上条件都不满足，说明两个矩形在 X、Y 方向都有重叠
    return true;
}
```

3.3 算法复杂度分析

该算法的时间复杂度为 $O(1)$ （常数时间），因为只需要进行最多 4 次比较操作。空间复杂度也是 $O(1)$ 。这是最简单高效的碰撞检测形式，也是 Unity 中 BoxCollider 检测能够高效运行的数学基础。

3.4 扩展：重叠深度计算

在实际物理仿真中，仅知道是否碰撞还不够，我们还需要知道重叠深度（Penetration Depth）以便将物体推开（分离/Resolution）。对于两个矩形，重叠深度可以计算如下：

Listing 3: 矩形重叠深度计算

```
// 返回两个矩形在各轴上的重叠深度
// overlapX > 0 表示 X 轴有重叠; overlapY > 0 表示 Y 轴有重叠
static void GetOverlap(Rectangle r1, Rectangle r2,
    out float overlapX, out float overlapY)
{
    overlapX = Math.Min(r1.Right, r2.Right) - Math.Max(r1.X, r2.X);
    overlapY = Math.Min(r1.Bottom, r2.Bottom) - Math.Max(r1.Y, r2.Y);
}
```

4 线段与三角形的相交检测

4.1 问题描述与应用场景

判断线段是否穿过三角形是一个非常常见且重要的问题。在实际应用中，当我们对模型进行射线检测（Raycast）时，实质上就是在判断一条射线（无限延伸的线段）与构成模型表面的许多三角形是否相交 [p.6]。

这个问题的核心包含两个子问题：

1. 求线段所在直线与三角形所在平面的交点（Intersection with Plane）
2. 判断该交点是否在三角形内部（Point-in-Triangle Test）

4.2 求线段与平面的交点

设线段两个端点为 $\mathbf{a}$ 和 $\mathbf{b}$ ，三角形三个顶点为 $\mathbf{v}_0$ 、 $\mathbf{v}_1$ 、 $\mathbf{v}_2$ [p.6]。

三角形的法向量 $\mathbf{n}$ 可以通过两条边的叉乘得到：

$$\mathbf{n} = (\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0)$$

对 $\mathbf{n}$ 进行归一化后，三角形所在平面的方程为：

$$\mathbf{n} \cdot (\mathbf{p} - \mathbf{v}_0) = 0$$

即平面上的任意一点 $\mathbf{p}$ 满足该点与 $\mathbf{v}_0$ 的连线垂直于法向量。

首先判断 $\mathbf{a}$ 和 $\mathbf{b}$ 是否在平面的同一侧：

- 分别计算 $\mathbf{a}$ 和 $\mathbf{b}$ 到平面的有符号距离（Signed Distance）：

$$d_a = \mathbf{n} \cdot (\mathbf{a} - \mathbf{v}_0), \quad d_b = \mathbf{n} \cdot (\mathbf{b} - \mathbf{v}_0)$$

- 如果 d_a 和 d_b 同号（符号相同），说明 $\mathbf{a}$ 和 $\mathbf{b}$ 都在平面的同一侧，则线段不穿过三角形所在的平面，必然不与三角形相交。
- 如果 d_a 和 d_b 异号（一个为正、一个为负），则线段穿过平面，交点 $\mathbf{x}$ 在线段 $\mathbf{ab}$ 上。

当 d_a 和 d_b 异号时，交点的位置可以通过线性插值（Linear Interpolation）求出 [p.6]。交点 $\mathbf{x}$ 将线段 $\mathbf{ab}$ 分为两段，比例与点到平面的有符号距离成正比：

$$\mathbf{x} = \frac{d_b \cdot \mathbf{a} - d_a \cdot \mathbf{b}}{d_b - d_a}$$

为什么这个等式成立？设交点为 $\mathbf{x} = \mathbf{a} + t(\mathbf{b} - \mathbf{a})$ ，其中 $t \in [0, 1]$ 是插值参数。由于 $\mathbf{x}$ 在平面上，满足 $\mathbf{n} \cdot (\mathbf{x} - \mathbf{v}_0) = 0$ ，代入可得：

$$\mathbf{n} \cdot (\mathbf{a} + t(\mathbf{b} - \mathbf{a}) - \mathbf{v}_0) = 0$$

$$\mathbf{n} \cdot (\mathbf{a} - \mathbf{v}_0) + t \cdot \mathbf{n} \cdot (\mathbf{b} - \mathbf{a}) = 0$$

$$d_a + t \cdot (d_b - d_a) = 0$$

$$t = -\frac{d_a}{d_b - d_a} = \frac{d_a}{d_a - d_b}$$

因此：

$$\mathbf{x} = \mathbf{a} + \frac{d_a}{d_a - d_b}(\mathbf{b} - \mathbf{a}) = \frac{(d_a - d_b)\mathbf{a} + d_a(\mathbf{b} - \mathbf{a})}{d_a - d_b} = \frac{-d_b\mathbf{a} + d_a\mathbf{b}}{d_a - d_b} = \frac{d_b \cdot \mathbf{a} - d_a \cdot \mathbf{b}}{d_b - d_a}$$

等号得证。

4.3 2D 中的点是否在三角形内：面积法

得到了线段与平面的交点 $\mathbf{x}$ 后，剩下的任务是判断 $\mathbf{x}$ 是否位于三角形 $\triangle v_0 v_1 v_2$ 的内部 [p.7]。

在 2D 空间中，一个非常直观的方法是面积法（Area Method）：将 $\mathbf{x}$ 分别与三角形的三个顶点连接，构成三个子三角形 $\triangle x v_1 v_2$ 、 $\triangle x v_2 v_0$ 、 $\triangle x v_0 v_1$ 。如果这三个子三角形的面积之和等于原三角形的面积，则 $\mathbf{x}$ 在三角形内部（或边界上）；如果不等于，则 $\mathbf{x}$ 在三角形外部。

三角形面积的计算可以通过向量叉乘完成。对于三个点 $A(a_x, a_y)$ 、 $B(b_x, b_y)$ 、 $C(c_x, c_y)$ ：

$$\text{Area}(A, B, C) = \frac{1}{2} |a_x(b_y - c_y) + b_x(c_y - a_y) + c_x(a_y - b_y)|$$

这本质上是向量 $\overrightarrow{AB}$ 和 $\overrightarrow{AC}$ 的叉乘（Cross Product）的模长的一半。

完整代码实现如下：

Listing 4: 面积法判断点在三角形内 (2D) (p.7)

```

/// <summary>
/// 计算三角形 ABC 的面积
/// </summary>
static float Area(Vector2 a, Vector2 b, Vector2 c)
{
    return Mathf.Abs(

```

```

        (a.x * (b.y - c.y) +
         b.x * (c.y - a.y) +
         c.x * (a.y - b.y)) / 2.0f
    );
}

/// <summary>
/// 判断点 p 是否在三角形 abc 内 (2D 面积法)
/// </summary>
public static bool IsPointInTriangle(
    Vector2 p, Vector2 a, Vector2 b, Vector2 c)
{
    float A = Area(a, b, c);          // 原三角形面积
    float A1 = Area(p, b, c);         // 子三角形 pbc
    float A2 = Area(p, c, a);         // 子三角形 pca
    float A3 = Area(p, a, b);         // 子三角形 pab
    // 面积和应等于原三角形面积 (允许浮点误差)
    return Mathf.Approximately(A, A1 + A2 + A3);
}

```

注意事项: 由于浮点数精度问题, 使用 `Mathf.Approximately` 或自定义一个小的容差值 ε (如 10^{-6}) 来做比较, 而不是直接进行严格相等判断。

4.4 3D 中的点是否在三角形内: 重心坐标法

在 3D 空间中, 面积法不方便直接使用 (因为点在三角形平面内, 但需要处理 3D 坐标)。更通用的方法是使用**重心坐标 (Barycentric Coordinates)** [p.8]。

在三角形 $\triangle ABC$ 中, 平面上的任意点 p 都可以表示为三个顶点的线性组合:

$$p = u \cdot a + v \cdot b + w \cdot c$$

其中 $u + v + w = 1$ 。 (u, v, w) 就是点 p 关于三角形 ABC 的重心坐标。

点 p 在三角形内部 (包括边界) 的充要条件是:

$$u \geq 0, \quad v \geq 0, \quad w \geq 0$$

如何求解重心坐标 (u, v, w) ? 假设我们已经知道 p 在三角形平面上 (通过前一步的平面交线计算得到), 可以利用向量投影求解。

设 $v_0 = b - a$, $v_1 = c - a$, $v_2 = p - a$ 。我们需要求 v 和 w 使得:

$$v_2 = v \cdot v_0 + w \cdot v_1$$

分别与 v_0 和 v_1 做点乘:

$$v_2 \cdot v_0 = v(v_0 \cdot v_0) + w(v_1 \cdot v_0)$$

$$v_2 \cdot v_1 = v(v_0 \cdot v_1) + w(v_1 \cdot v_1)$$

记：

$$\begin{aligned} d_{00} &= \mathbf{v}_0 \cdot \mathbf{v}_0 & d_{01} &= \mathbf{v}_0 \cdot \mathbf{v}_1 \\ d_{11} &= \mathbf{v}_1 \cdot \mathbf{v}_1 & d_{20} &= \mathbf{v}_2 \cdot \mathbf{v}_0 \\ d_{21} &= \mathbf{v}_2 \cdot \mathbf{v}_1 \end{aligned}$$

则得到线性方程组：

$$\begin{cases} d_{20} = v \cdot d_{00} + w \cdot d_{01} \\ d_{21} = v \cdot d_{01} + w \cdot d_{11} \end{cases}$$

使用克莱姆法则（Cramer's Rule）求解：

$$\begin{aligned} \text{denom} &= d_{00} \cdot d_{11} - d_{01}^2 \\ v &= \frac{d_{11} \cdot d_{20} - d_{01} \cdot d_{21}}{\text{denom}}, \quad w = \frac{d_{00} \cdot d_{21} - d_{01} \cdot d_{20}}{\text{denom}}, \quad u = 1 - v - w \end{aligned}$$

完整代码实现：

Listing 5: 重心坐标法判断点在三角形内 (3D) (p.8)

```

/// <summary>
/// 计算点 p 关于三角形 abc 的重心坐标
/// </summary>
static void Barycentric(Vector3 p,
    Vector3 a, Vector3 b, Vector3 c,
    out float u, out float v, out float w)
{
    Vector3 v0 = b - a, v1 = c - a, v2 = p - a;
    float d00 = Vector3.Dot(v0, v0);
    float d01 = Vector3.Dot(v0, v1);
    float d11 = Vector3.Dot(v1, v1);
    float d20 = Vector3.Dot(v2, v0);
    float d21 = Vector3.Dot(v2, v1);
    float denom = d00 * d11 - d01 * d01;
    v = (d11 * d20 - d01 * d21) / denom;
    w = (d00 * d21 - d01 * d20) / denom;
    u = 1.0f - v - w;
}

/// <summary>
/// 判断点 p 是否在三角形 abc 内 (3D 重心坐标法)
/// </summary>
public static bool IsPointInTriangle(
    Vector3 p, Vector3 a, Vector3 b, Vector3 c)
{
    float u, v, w;
    Barycentric(p, a, b, c, out u, out v, out w);
}

```

```

    return (u >= 0) && (v >= 0) && (w >= 0);
}

```

4.5 完整流程：线段与三角形的 3D 相交检测

将以上两个子步骤组合起来，得到完整的线段-三角形相交检测算法：

Listing 6: 线段与三角形相交检测（完整）

```

/// <summary>
/// 判断线段 ab 是否与三角形 (v0,v1,v2) 相交,
/// 若相交, 返回交点 hitPoint
/// </summary>
public static bool RayTriangleIntersection(
    Vector3 a, Vector3 b,
    Vector3 v0, Vector3 v1, Vector3 v2,
    out Vector3 hitPoint)
{
    hitPoint = Vector3.zero;

    // 1. 计算三角形法向量
    Vector3 n = Vector3.Cross(v1 - v0, v2 - v0).normalized;

    // 2. 判断线段两端是否在平面同一侧
    float da = Vector3.Dot(n, a - v0);
    float db = Vector3.Dot(n, b - v0);

    // 同号则不相交（或与平面平行偏移）
    if (da * db > 0) return false;

    // 3. 计算线段与平面的交点
    float t = da / (da - db);
    Vector3 x = a + t * (b - a);    // 交点

    // 4. 判断交点是否在三角形内（重心坐标法）
    float u, v, w;
    Barycentric(x, v0, v1, v2, out u, out v, out w);

    if (u >= 0 && v >= 0 && w >= 0)
    {
        hitPoint = x;
        return true;
    }
    return false;
}

```

5 线段与模型的相交检测

5.1 模型即三角形的集合

在 Unity（以及几乎所有 3D 图形引擎）中，**模型（Model）**是由大量三角形组成的**网格（Mesh）**。Mesh 的基本数据包括顶点数组（Vertices）和三角形索引数组（Triangles），每三个连续的索引构成一个三角形面 [p.9]。

既然我们已经掌握了判断线段与单个三角形是否相交的算法，那么理论上，判断线段与模型是否相交的方法也很直接：

对模型中的每一个三角形，依次使用上述算法进行相交检测。只要有一个三角形与线段相交，即可判定线段与模型相交。

从功能上看，这是正确的。可以通过遍历 Mesh.triangles 数组来逐三角形检测。

5.2 性能问题

然而，上述朴素方法存在严重的性能问题。一个中等复杂度的 3D 模型通常包含数千甚至数万个三角形。例如：

- 一个简单的角色模型：约 5,000–15,000 个三角形
- 一个详细的建筑模型：约 20,000–100,000 个三角形
- 大型场景模型：可达数百万个三角形

在游戏这种需要 60 FPS（每帧约 16.7ms）的实时应用场景中，对每个射线检测遍历模型的所有三角形是完全不可接受的。这正是下一节要讨论的**加速结构（Acceleration Structures）**所要解决的问题。

在 Unity 中，引擎内部已经实现了高效的 BVH 加速结构，因此使用 `Physics.Raycast()` 的性能是可以接受的。但如果我们自己实现程序化物理系统，就需要了解这些优化方法。

6 三角形与三角形的碰撞检测

6.1 问题描述

三角形之间的碰撞检测在实际应用中无处不在 [p.10]。两个模型之间的碰撞，本质上就是两个模型的三角形集合之间的碰撞判断。一个大概的思路是：

- 对于三角形 T_1 的三个顶点 v_0 、 v_1 、 v_2 ，分别判断它们位于三角形 T_2 的哪一侧（利用 T_2 的法向量）。
- 如果三个顶点都在 T_2 的同一侧且与 T_2 平面有距离，则 T_1 和 T_2 不相交。
- 否则，计算出可能的接触线，并判断该线段是否在 T_1 和 T_2 内部。

- 同时还需要对称地检查 T_2 的三个顶点是否穿过 T_1 的平面。

具体的实现比较繁琐。经典参考论文为：

"A Fast Triangle-Triangle Intersection Test"

— Tomas Moller, Journal of Graphics Tools, 2(2), 1997.

该论文提出的算法是业界标准，广泛应用于碰撞检测库中（如 Bullet Physics、PhysX 等）。

有了三角形与三角形的碰撞检测方法，理论上就可以实现任意两个模型之间的碰撞检测了：只需对两个模型的所有三角形做两两比较。当然，这在实际中是不可行的（ $O(n \times m)$ 的复杂度），因此需要配合下文将要介绍的 BVH 加速结构。

7 包围体积层级 (BVH)

7.1 为什么需要 BVH

如前所述，在处理模型级别的碰撞检测时，如果对其中的每个三角形都逐一检测，性能开销极其巨大，在实时游戏中完全不可行 [p.12]。

包围体积层级 (Bounding Volume Hierarchy, BVH) 正是为了解决这个问题而设计的加速结构。这些结构的核心思想是一致的：

与其直接考虑每一个细小的三角形，不如先考虑包含它们的一组较大的包围体积 (Bounding Volume)。如果连这个较大的体积都没有发生碰撞，那么它内部包含的所有三角形就完全不需要进一步检测了。

这种「从大到小」的层级结构形成了一个**树 (Tree)**：根节点 (Root) 包含整个模型的大包围盒；叶子节点 (Leaf) 包含少量（甚至一个）三角形；中间节点 (Internal Node) 逐层细分。

7.2 BVH 的工作原理

当需要进行碰撞检测时：

1. 从根节点的大包围盒开始检测
2. 如果射线/物体与该包围盒不相交，则立即跳过该节点下的所有子节点（这是一次性排除大量三角形的关键）
3. 如果相交，则继续向子节点深入，直到到达叶子节点
4. 在叶子节点中，对其中的少量三角形进行精确检测

这种层级遍历策略将时间复杂度从 $O(n)$ （遍历所有三角形）降低到大约 $O(\log n)$ （只遍历树中与射线路径相交的节点）。

常见的 BVH 结构包括 [p.12]：

- **Octree**（八叉树）：每个节点有 8 个子节点，将 3D 空间均匀划分为八个等大的子立方体
- **K-D Tree**（K 维树）：二叉树结构，每个节点在某个坐标轴方向上将空间一分为二
- **BSP Tree**（二叉空间分割树，**Binary Separating Planes**）：使用任意方向的平面分割空间，比 K-D Tree 更灵活但构建更复杂

7.3 射线与 BVH 的相交检测：伪代码

以下是一段通用的射线-BVH 相交检测伪代码。无论使用哪一种 BVH 实现（Octree、K-D Tree 或 BSP），大致的逻辑结构都是相似的 [p.13]：

Listing 7: 射线-BVH 相交检测伪代码 (p.13)

```
function IntersectRayWithBVH(ray, bvh):
    # 初始化待检测节点列表
    toVisit = []
    toVisit.append(bvh.root)

    while toVisit is not empty:
        node = toVisit.pop() # 取出最后一个节点

        # Step 1: 检查射线是否与节点的包围盒相交
        if not ray.Intersects(node.boundingBox):
            continue # 不相交，跳过该节点及其所有子节点

        # Step 2: 如果是叶子节点
        if node.isLeaf:
            for each object in node.objects:
                if ray.Intersects(object):
                    return True # 或返回碰撞的物体和交点

        # Step 3: 如果是内部节点
        else:
            toVisit.append(node.leftChild)
            toVisit.append(node.rightChild)

    return False # 遍历完毕，没有发现碰撞
```

这段伪代码体现了 BVH 遍历的核心思想：

1. 使用一个栈（toVisit 列表）来管理待检测的节点
2. 对每个节点首先做一次廉价的包围盒-射线检测（Broad Phase），快速排除不相交的节点
3. 只有无法排除的节点才深入检查，最终在叶子节点中进行精确检测（Narrow Phase）

7.4 Octree 八叉树

八叉树 (Octree) 是最常见的空间划分优化结构之一。它在处理大规模 3D 数据时非常有效, 例如 Minecraft 等沙盒游戏就使用八叉树来管理区块数据 [p.14]。

核心概念: Octree 是一种每个非叶子节点拥有恰好 8 个子节点的树状结构。

在 3D 世界中, 这对应了将一个立方体 (Cube) 沿着 X、Y、Z 三个方向的中心面切成 8 个等大的小立方体, 每个小立方体由对应的子节点管理。因为划分了三个维度、每个维度一分为二, 总共是 $2^3 = 8$ 个子空间, 所以叫「八叉」树。

为什么是 8? : 这源自于三维空间的维度特性。可以类似地理解:

- 1D 空间: 二叉树 (Binary Tree), 每次将线段一分为二 ($2^1 = 2$)
- 2D 空间: 四叉树 (Quadtree), 每次将正方形一分为四 ($2^2 = 4$)
- 3D 空间: 八叉树 (Octree), 每次将立方体一分为八 ($2^3 = 8$)

不存在「九叉树」这种东西, 正是因为一个立方体被三个相互垂直的面对称划分后, 只能产生 8 个子块。

射线检测中的使用: 当使用 Octree 加速射线检测时, 从根节点 (包含整个场景的最大立方体) 开始, 判断射线穿过了哪几个子立方体。只对被穿过的子立方体继续递归检测, 从而在检测过程中快速排除大量不会发生碰撞的空间区域。

优缺点:

- 优点: 实现简单, 每个节点的子节点数量固定 (8), 内存布局规整, 适合 GPU 并行遍历。
- 缺点: 空间划分是均匀的, 对于物体分布不均匀的场景 (如大量物体聚集在某一侧) 效率不高, 可能产生大量空节点。

7.5 K-D Tree

K-D Tree (K-Dimensional Tree, K 维树) 是一种管理 K 维空间中数据点的数据结构 [p.15]。

核心概念: 与 Octree 不同, K-D Tree 采用二叉树结构。每个节点选择一个坐标轴 (X、Y 或 Z), 在该轴上的某个位置将空间一分为二。两个子区域再分别选择坐标轴继续分割, 依此类推。

例如, 在 2D K-D Tree 中:

- 根节点 A 沿 X 轴方向, 将整个平面分为左、右两个半平面
- 左半平面的节点 B 沿 Y 轴方向, 将左半平面分为上、下两部分
- 以此类推, 逐层交替选择分割轴

K-D Tree 的特点 [p.15]:

1. 每条分割边都是坐标轴对齐的（**Axis-Aligned**）：分割面始终垂直于某一个坐标轴，保证划分简单高效。
2. 每个分割点对应一个实际的数据点：不是随意选择分割位置，而是在实际存在的数据点处进行划分。这保证了树中的每个节点都存储有意义的数据。
3. 每一个数据点只包含在一个节点内：不存在数据点在多个节点中重复存储的问题，保证了空间效率。

与 Octree 的对比：

特性	Octree	K-D Tree
树结构	八叉树（每节点 8 子节点）	二叉树（每节点 2 子节点）
分割方式	在空间中心均匀分割（三个维度同时）	在数据点处沿单一轴分割（每次一个维度）
空间利用率	均匀分布时好，稀疏时浪费	自适应数据分布，空间利用率高
分割面	固定位于包围盒的中心	可以位于任意数据点所在位置
实现难度	相对简单	构建逻辑稍复杂（需选择分割策略）
适用场景	GPU 并行遍历、均匀空间划分	点云数据、非均匀分布的场景

表 1: Octree 与 K-D Tree 对比

8 Unity LineRenderer 组件

8.1 组件概述

LineRenderer（线段渲染器）是 Unity 提供的一个便捷组件，用于在场景中显示各种线形物体，如激光束、绳索、拖尾效果、手绘线条等 [p.18]。

LineRenderer 的工作原理是：给定一系列 3D 空间中的点（位置），Unity 自动在这些点之间绘制连续的线段，将它们连成一条线。

8.2 核心属性：Positions

LineRenderer 最重要的属性是 `positions`，它是一个 `Vector3[]` 数组，存储了这条线段上所有端点（Knots/Control Points）的位置 [p.18]。

- `positions[0]`: 线段的起点
- `positions[1]`: 线段的第一个中间点
- ...
- `positions[n-1]`: 线段的终点

在 Inspector 面板中可以直接编辑 `positions` 数组，也可以通过代码动态修改。常见用法：

Listing 8: LineRenderer 基本使用

```
using UnityEngine;

public class LineExample : MonoBehaviour
{
    private LineRenderer lineRenderer;

    void Start()
    {
        lineRenderer = GetComponent<LineRenderer>();
        // 设置线的宽度
        lineRenderer.startWidth = 0.1f;
        lineRenderer.endWidth = 0.1f;
        // 设置位置点
        lineRenderer.positionCount = 3;
        lineRenderer.SetPosition(0, new Vector3(0, 0, 0));
        lineRenderer.SetPosition(1, new Vector3(1, 2, 0));
        lineRenderer.SetPosition(2, new Vector3(3, 1, 0));
    }

    void Update()
    {
        // 在运行时动态添加点
        if (Input.GetMouseButtonDown(0))
        {
            Vector3 mousePos = Camera.main.ScreenToWorldPoint(
                new Vector3(Input.mousePosition.x, Input.mousePosition.y, 10))
            ;
            mousePos.z = 0;
            int count = lineRenderer.positionCount;
            lineRenderer.positionCount = count + 1;
            lineRenderer.SetPosition(count, mousePos);
        }
    }
}
```

8.3 LineRenderer 的实用属性

除了 positions 之外, LineRenderer 还有一些常用属性:

- startWidth / endWidth: 线段的起始宽度和结束宽度 (可实现渐细效果)
- startColor / endColor: 线段的起始颜色和结束颜色 (可实现渐变色效果)
- material: 线段使用的材质
- useWorldSpace: 是否使用世界坐标 (true) 还是本地坐标 (false)

- `loop`: 是否将首尾相连形成闭环
- `numCornerVertices`: 拐角处的圆滑程度（增加顶点使拐角更圆润）
- `numCapVertices`: 端点处的圆滑程度

9 Unity PolygonCollider2D 组件

9.1 组件概述

PolygonCollider2D（多边形碰撞器）是 Unity 2D 物理系统中的一种碰撞器类型。当我们给一个 Sprite（精灵图片）添加 PolygonCollider2D 时，Unity 会自动分析图片的轮廓，生成一个紧密贴合图片形状的多边形碰撞区域 [p.19]。

这种自动拟合（Auto-Fitting）功能就是通过 PolygonCollider2D 组件的多边形顶点集实现的。

9.2 核心属性：Points

与 LineRenderer 的 `positions` 非常相似，PolygonCollider2D 最重要的属性是 `points`，它是一个 `Vector2[]` 数组，存储了多边形所有的顶点 [p.19]。

- `points` 定义的多边形必须是**封闭的（Closed）**：首尾相连构成一个完整的多边形
- 顶点顺序为**顺时针（Clockwise）**或**逆时针（Counter-Clockwise）**
- 多边形必须是**简单的（Simple）**：边之间不能自交
- Unity 要求多边形的顶点数至少为 3

9.3 LineRenderer 与 PolygonCollider2D 的结合

注意到 LineRenderer 的 `positions` 和 PolygonCollider2D 的 `points` 都是点数组，这启发了一个自然的想法 [p.19]：

将 LineRenderer 中存储的线段端点转换为 PolygonCollider2D 的多边形顶点，从而生成一个贴合线段形状的碰撞器。

这样就能实现「画一条有物理效果的线」：用户在屏幕上画线，系统自动为这条线生成碰撞形状，使得其他物体可以与这条线发生物理交互。

10 从线段生成贴合碰撞器

10.1 核心问题

关键的技术挑战在于 [p.20]：如何将一个没有宽度的、开放的点集（LineRenderer 的 `positions`）转化为一个有宽度、有面积的封闭点集（PolygonCollider2D 的 `points`）。

LineRenderer 的 positions 只是一条线上的离散采样点，它们定义了一条有方向（从左到右或从右到左）的路径。要将其转换为一个封闭的多边形，需要用这条路径的**两边扩展**出一个有宽度的带状区域。

10.2 算法：通过线段扩展生成多边形

核心思路：对于路径上相邻的每一对点 (P_i, P_{i+1}) ，计算出线段 $P_i P_{i+1}$ 的法向量方向，然后沿法向量正方向和负方向各偏移一定距离（即线宽度的一半），得到两个偏移点。将一侧的所有偏移点连起来形成多边形的「上边」，另一侧的所有偏移点连起来形成「下边」，再将两端封口，即得到一个封闭的多边形。

详细步骤：

Step 1. **计算线段方向：**对于第 i 段（从 P_i 到 P_{i+1} ），方向向量为

$$\vec{d}_i = P_{i+1} - P_i$$

Step 2. **计算线段法向量：**在 2D 中，一个向量 (x, y) 的垂直法向量是 $(-y, x)$ （或 $(y, -x)$ ）。归一化后得到单位法向量 $\vec{n}_i$ ：

$$\vec{n}_i = \frac{(-d_i.y, d_i.x)}{\|\vec{d}_i\|}$$

Step 3. **沿法向量偏移：**以线宽 w 的一半为偏移量，生成两侧的偏移点：

$$\begin{aligned} \text{Left}_i^{\text{top}} &= P_i + \frac{w}{2} \cdot \vec{n}_i \\ \text{Left}_i^{\text{bottom}} &= P_i - \frac{w}{2} \cdot \vec{n}_i \\ \text{Right}_i^{\text{top}} &= P_{i+1} + \frac{w}{2} \cdot \vec{n}_i \\ \text{Right}_i^{\text{bottom}} &= P_{i+1} - \frac{w}{2} \cdot \vec{n}_i \end{aligned}$$

Step 4. **处理拐角：**在相邻线段的连接处（拐角），两条线段的法向量方向不同，偏移点不会重合。需要计算两条偏移线的交点（Miter Joint），或者简单地使用倒角（Bevel Joint）或圆弧（Round Joint）过渡。

Step 5. **组合成封闭多边形：**将所有「上边」偏移点按顺序排列，然后逆序排列所有「下边」偏移点，形成封闭多边形。

10.3 完整代码实现

以下是一个在 Unity 中实现从 LineRenderer 位置生成 PolygonCollider2D 的完整示例：

Listing 9: 从 LineRenderer 生成 PolygonCollider2D

```
using UnityEngine;
using System.Collections.Generic;

[RequireComponent(typeof(LineRenderer))]
```

```
[RequireComponent(typeof(PolygonCollider2D))]  
public class LineToCollider : MonoBehaviour  
{  
    [SerializeField] private float lineWidth = 0.2f;  
  
    private LineRenderer lineRenderer;  
    private PolygonCollider2D polygonCollider;  
  
    void Start()  
    {  
        lineRenderer = GetComponent<LineRenderer>();  
        polygonCollider = GetComponent<PolygonCollider2D>();  
    }  
  
    void Update()  
    {  
        UpdateCollider();  
    }  
  
    /// <summary>  
    /// 根据当前 LineRenderer 的 positions 更新 PolygonCollider2D 的 points  
    /// </summary>  
    public void UpdateCollider()  
    {  
        int pointCount = lineRenderer.positionCount;  
        if (pointCount < 2) return; // 至少需要两个点才能形成线段  
  
        // 收集线段上的所有点  
        Vector3[] positions3D = new Vector3[pointCount];  
        lineRenderer.GetPositions(positions3D);  
  
        // 转换为 2D 点  
        Vector2[] path = new Vector2[pointCount];  
        for (int i = 0; i < pointCount; i++)  
            path[i] = new Vector2(positions3D[i].x, positions3D[i].y);  
  
        // 生成两侧的偏移点  
        List<Vector2> topPoints = new List<Vector2>();  
        List<Vector2> bottomPoints = new List<Vector2>();  
  
        float halfWidth = lineWidth / 2f;  
  
        for (int i = 0; i < pointCount - 1; i++)  
        {  
            Vector2 dir = (path[i + 1] - path[i]).normalized;  
            Vector2 normal = new Vector2(-dir.y, dir.x); // 2D法向量
```

```

        Vector2 topPoint = path[i] + normal * halfWidth;
        Vector2 bottomPoint = path[i] - normal * halfWidth;

        topPoints.Add(topPoint);
        bottomPoints.Add(bottomPoint);
    }

    // 添加最后一个点的偏移
    {
        int last = pointCount - 1;
        Vector2 dir = (path[last] - path[last - 1]).normalized;
        Vector2 normal = new Vector2(-dir.y, dir.x);
        topPoints.Add(path[last] + normal * halfWidth);
        bottomPoints.Add(path[last] - normal * halfWidth);
    }

    // 组合成封闭多边形
    List<Vector2> polygonPoints = new List<Vector2>();
    polygonPoints.AddRange(topPoints); // 上边（正向）
    bottomPoints.Reverse(); // 反转下边
    polygonPoints.AddRange(bottomPoints); // 下边（逆向）

    // 设置 PolygonCollider2D
    polygonCollider.SetPath(0, polygonPoints.ToArray());
}
}

```

10.4 拐角处理详解

上述简单实现中，每个点处的拐角是两个偏移点的直线连接。当线段拐弯较急锐时，这可能导致碰撞器在拐角处覆盖不足。

更完善的处理方式包括：

- **Miter Joint（斜接）**：计算两条相邻线段偏移线的交点，用交点作为拐角顶点。当拐角太尖锐时可能导致交点无限远（需设上限）。
- **Bevel Joint（倒角）**：直接用一条边连接两条偏移线的端点。简单但拐角处不够圆滑。
- **Round Joint（圆弧）**：在拐角处用圆弧连接，视觉效果最好但顶点数较多。

10.5 运行时实时更新

课程要求之一是在运行时实时更新线段以及碰撞盒的形状 [p.20]。这通过在 `Update()` 方法中持续调用 `UpdateCollider()` 来实现（如上例所示）。

每次更新时，LineRenderer 的 positions 可能因为用户输入、动画或游戏逻辑而改变，碰撞器也随之刷新，保持与线段形状的同步。

11 课后习题详解

11.1 习题 1：使用叉乘判断点是否在三角形内

题目：我们可以使用向量的叉乘来判断一个点是否在三角形内。请查阅相关资料并编写一个使用叉乘判断点是否在三角形内的函数。[p.21]

解法思路：

叉乘法（Cross Product Method）是除面积法和重心坐标法之外的第三种判断点在三角形内的方法。其核心原理是：

设三角形 ABC 和待判断点 P 。分别计算以下三个叉乘结果（在 2D 中，叉乘结果是标量）：

$$\text{Cross}_0 = (B - A) \times (P - A)$$

$$\text{Cross}_1 = (C - B) \times (P - B)$$

$$\text{Cross}_2 = (A - C) \times (P - C)$$

其中二维叉乘定义为： $(x_1, y_1) \times (x_2, y_2) = x_1y_2 - x_2y_1$ 。

如果三个叉乘结果的符号全部相同（全为正或全为负），则点 P 在三角形内部。其几何含义是：点 P 始终位于三角形每条边的同一侧（顺时针或逆时针方向）。

注意：需要确保三角形的顶点顺序是一致的（要么全都是顺时针，要么全都是逆时针）。如果顶点顺序不确定，可以取第一个叉乘的符号作为参考，或者使用绝对值比较。

完整代码：

Listing 10: 习题 1：叉乘法判断点在三角形内

```

/// <summary>
/// 二维叉乘：(a.x, a.y) x (b.x, b.y) = a.x*b.y - a.y*b.x
/// 结果 > 0: b 在 a 的逆时针方向
/// 结果 < 0: b 在 a 的顺时针方向
/// 结果 = 0: a 与 b 共线
/// </summary>
static float Cross(Vector2 a, Vector2 b)
{
    return a.x * b.y - a.y * b.x;
}

/// <summary>
/// 使用叉乘法判断点 p 是否在三角形 (v0, v1, v2) 内
/// </summary>
public static bool IsPointInTriangleCross(
    Vector2 p, Vector2 v0, Vector2 v1, Vector2 v2)

```

```

{
    // 判断三角形方向：计算整个三角形的有向面积
    float area2 = Cross(v1 - v0, v2 - v0);
    bool isClockwise = (area2 < 0);

    // 对每条边，计算 p 在边的哪一侧
    float c0 = Cross(v1 - v0, p - v0); // 边 v0->v1
    float c1 = Cross(v2 - v1, p - v1); // 边 v1->v2
    float c2 = Cross(v0 - v2, p - v2); // 边 v2->v0

    if (isClockwise)
    {
        // 三角形顺时针：三个叉乘都应 <= 0
        return (c0 <= 0) && (c1 <= 0) && (c2 <= 0);
    }
    else
    {
        // 三角形逆时针：三个叉乘都应 >= 0
        return (c0 >= 0) && (c1 >= 0) && (c2 >= 0);
    }
}

/// <summary>
/// 另一种实现：不考虑三角形方向，使用相对判断
/// </summary>
public static bool IsPointInTriangleCrossV2(
    Vector2 p, Vector2 v0, Vector2 v1, Vector2 v2)
{
    // 计算 p 相对于每条边的位置
    float d1 = Cross(v1 - v0, p - v0);
    float d2 = Cross(v2 - v1, p - v1);
    float d3 = Cross(v0 - v2, p - v2);

    // 检查是否全部同号（全正或全负）
    bool hasNeg = (d1 < 0) || (d2 < 0) || (d3 < 0);
    bool hasPos = (d1 > 0) || (d2 > 0) || (d3 > 0);

    // 不能同时有正有负（允许为 0，即点在边上）
    return !(hasNeg && hasPos);
}

```

比较三种方法：

特性	面积法	重心坐标法	叉乘法
维度	2D	2D/3D	2D
浮点精度	面积累加有误差	精度较好	精度最好（直接判断符号）
计算复杂度	低（3 个面积）	中（解 2x2 方程）	最低（3 个叉乘）
额外信息	无	给出插值权重 (u,v,w)	可判断在边的哪一侧

表 2: 三种点在三角形内判断方法的对比

11.2 习题 2：圆与矩形的碰撞检测

题目：编写一个判断圆与矩形是否相交的函数。[p.21]

解法思路：

圆与矩形的碰撞检测可以通过以下步骤实现：

1. 找到矩形上距离圆心最近的点（Closest Point）
2. 计算该最近点与圆心之间的距离
3. 如果距离小于等于圆的半径，则发生碰撞

找矩形上距离圆心最近点的关键：将圆心坐标的每个分量分别 Clamp（钳制）到矩形的对应范围内。

完整代码：

Listing 11: 习题 2：圆与矩形碰撞检测

```

/// <summary>
/// 矩形数据结构
/// </summary>
public struct Rect2D
{
    public Vector2 center;    // 矩形中心
    public Vector2 halfSize; // 半宽半高

    public float Left    { get { return center.x - halfSize.x; } }
    public float Right   { get { return center.x + halfSize.x; } }
    public float Top     { get { return center.y + halfSize.y; } }
    public float Bottom  { get { return center.y - halfSize.y; } }

    public Rect2D(Vector2 center, Vector2 halfSize)
    {
        this.center = center;
        this.halfSize = halfSize;
    }
}

```

```
/// <summary>
/// 圆数据结构
/// </summary>
public struct Circle
{
    public Vector2 center;
    public float radius;

    public Circle(Vector2 center, float radius)
    {
        this.center = center;
        this.radius = radius;
    }
}

/// <summary>
/// 判断圆与矩形是否相交
/// </summary>
public static bool CircleRectIntersect(Circle circle, Rect2D rect)
{
    // Step 1: 找到矩形上距离圆心最近的点
    float closestX = Mathf.Clamp(circle.center.x,
        rect.Left, rect.Right);
    float closestY = Mathf.Clamp(circle.center.y,
        rect.Bottom, rect.Top);

    Vector2 closestPoint = new Vector2(closestX, closestY);

    // Step 2: 计算圆心到最近点的距离
    float distSqr = (circle.center - closestPoint).sqrMagnitude;

    // Step 3: 比较距离与半径
    return distSqr <= circle.radius * circle.radius;
}
```

算法分析:

- 如果圆心在矩形内部, 最近点就是圆心本身, 距离为 0, 必然相交
- 如果圆心在矩形外部但在某一轴的投影范围内 (即圆心位于矩形的「上下带」或「左右带」), 最近点就是投影点
- 如果圆心在矩形的角区域 (即在两个轴的投影都超出范围), 最近点就是矩形的对应角点

特殊情况讨论:

- 矩形退化为线段 (宽度或高度为 0): 以上算法仍然正确运行

- 圆与矩形边相切 (Tangent): 使用 $\leq$ 判断, 切线情况算作相交 (碰撞)
- 浮点精度: 使用平方距离 (`sqrMagnitude`) 比较而非开方后的距离, 避免开方运算

11.3 习题 3: 线段点集的性能优化

题目: 在碰撞盒生成的项目中, 我们会实时生成并更新线段上的点。不过这样会有性能隐患: 线段上的点会越来越多, 性能压力会很快地失去控制。请问我们可以如何优化它呢? (提示: 一种方法是降低线段的「分辨率」) [p.21]

问题分析:

当用户持续画线时, `LineRenderer` 的 `positions` 数组中的点数量会不断增长。每次 `Update` 都需要遍历所有点来重新生成 `PolygonCollider2D` 的顶点, 时间复杂度为 $O(n)$ (n 为点数)。随着 n 不断增大, 每帧的 CPU 开销也会线性增长, 最终导致帧率下降。

此外, `PolygonCollider2D` 的点数过大也会影响物理引擎的碰撞检测性能。Unity 的物理引擎对多边形碰撞器的处理复杂度与顶点数相关。

优化方案:

方案 1. 降采样 / 降低分辨率 (Downsampling)

最直接的优化方法: 不必保留线段上的每一个点, 而是只保留关键点。常见策略包括:

- 每隔 N 个点采样一个 (均匀降采样): 例如每 5 个点保留 1 个, 将点数减少到原来的 $1/5$ 。
- 基于距离的降采样: 只有当新点与上一个保留点的距离超过阈值 $D_{\min}$ 时才添加该点。这样在画得慢时自然保持高分辨率, 画得快时自动降低密度。
- 基于角度的降采样: 只有当线段方向变化超过一定角度时, 才保留中间点。在直线段上只需要两个端点, 在拐弯处保留更多点以保持形状。

代码示例:

Listing 12: 基于距离的降采样

```
/// <summary>
/// 对路径点进行基于最小距离的降采样
/// </summary>
public static List<Vector2> DownsampleByDistance(
    List<Vector2> path, float minDistance = 0.5f)
{
    if (path.Count <= 2) return new List<Vector2>(path);

    List<Vector2> result = new List<Vector2>();
    result.Add(path[0]);           // 始终保留起点
    Vector2 lastKept = path[0];

    for (int i = 1; i < path.Count - 1; i++)
```

```
{  
    float dist = Vector2.Distance(lastKept, path[i]);  
    if (dist >= minDistance)  
    {  
        result.Add(path[i]);  
        lastKept = path[i];  
    }  
}  
result.Add(path[path.Count - 1]); // 始终保留终点  
  
return result;  
}
```

方案 2. 增量更新 (Incremental Update)

不必每帧重新生成整个碰撞器。只更新变化的部分：

- 当用户在线段末尾添加新点时，只计算最后一个线段段的碰撞器顶点并追加到碰撞器点集中
- 当线段整体缩放或移动时，使用 Transform 的变换而非重新计算碰撞器

方案 3. 分块管理 (Chunking)

将长线段分割成多个固定长度的段 (Chunk)，每个段拥有自己的 LineRenderer 和 PolygonCollider2D。这样：

- 更新时只需要更新最后一个活跃段
- 远处的不变段可以标记为静态 (Static)，Unity 会对静态碰撞器做更好的优化
- 每个段的碰撞器顶点数有上限，保证单次更新的性能可预测

方案 4. 使用 EdgeCollider2D 替代 PolygonCollider2D

对于开放的线段，EdgeCollider2D 比 PolygonCollider2D 更合适。EdgeCollider2D 不需要形成封闭多边形，只需要定义边缘的顶点即可，顶点数是 PolygonCollider2D 的一半左右。

方案 5. LOD (Level of Detail) 策略

根据线段与摄像机的距离，使用不同分辨率的碰撞器：

- 近距离 (High LOD)：高精度碰撞器，用于精确的物理交互
- 中距离 (Medium LOD)：使用降采样后的碰撞器
- 远距离 (Low LOD)：使用 AABB 或最简单的外包围盒，甚至完全禁用碰撞

方案 6. 限制总点数上限

设置一个绝对上限（如最多 500 个点）。当超过上限时，移除最早的点（FIFO 队列），类似于一个固定长度的滑动窗口。这样可以保证无论用户画多久，性能都稳定在一个可预测的水平。

Listing 13: 固定上限的滑动窗口

```

/// <summary>
/// 维护一个固定大小的路径点队列
/// </summary>
public class FixedSizePath
{
    private List<Vector2> points = new List<Vector2>();
    private int maxPoints;

    public FixedSizePath(int maxPoints)
    {
        this.maxPoints = maxPoints;
    }

    public void AddPoint(Vector2 point)
    {
        points.Add(point);
        // 超过上限时移除最早的点
        while (points.Count > maxPoints)
            points.RemoveAt(0);
    }

    public List<Vector2> GetPoints() => points;
}

```

优化方案对比:

方案	优点	缺点	适用场景
降采样	实现简单，效果明显	可能丢失细节	平滑曲线
增量更新	理论上最优	实现复杂，需处理多种情况	线性追加场景
分块管理	性能可预测	架构较复杂	超长线段
EdgeCollider2D	顶点数减半	仍需管理点数增长	开放线段
LOD 策略	远近兼顾	实现复杂	3D 场景
固定上限	最可靠保证	旧线段会消失	临时涂鸦

表 3: 线段点集性能优化方案对比

12 总结

本课深入讲解了程序化物理与碰撞盒生成的核心知识体系:

1. **碰撞检测算法基础:** 从最简单的矩形碰撞到复杂的三角形-三角形碰撞，掌握了分离轴定理、平面交点计算、点在三角形内判断（面积法、重心坐标法、叉乘法）等经典算法。

2. **加速结构 BVH:** 理解了 Octree 和 K-D Tree 这两种最常用的空间划分结构。掌握了射线-BVH 相交检测的通用伪代码逻辑，以及不同树结构的适用场景。
3. **Unity 组件:** 熟练掌握了 LineRenderer 和 PolygonCollider2D 的核心属性 (positions/points) 和使用方法。
4. **程序化碰撞生成:** 学会了如何将一个开放的点集转换为封闭的、有宽度的多边形碰撞器，解决了「给线加上物理」这一实际开发需求。
5. **性能优化思维:** 通过分析线段点数无限增长的性能隐患，了解了降采样、增量更新、分块管理等实用优化策略。

这些知识不仅适用于 Unity 开发，在其他游戏引擎 (Unreal Engine、Godot 等) 和一般的计算几何场景中同样具有重要价值。